

Arquitectura, Diseño Técnico y Pruebas del Backend

Sistema de Gestión de Citas para Tutorías IQ English

Human-Computer Interaction, and Digital Citizenship (CSE6061)
Cuevas, Ramon

30 SEPTIEMBRE 2026

Arquitectura, Diseño Técnico y Pruebas del Backend.

Ficha Técnica del Componente Backend.

La Tabla 1 que se presenta a continuación contiene la información detallada de las tecnologías que están involucradas en la implementación del componente Backend del Sistema de Gestión de Citas para Tutorías IQ English.

Parámetro / Componente	Descripción y Especificación Técnica
Nombre del Proyecto	IQ English Tutoring Management Backend Platform
Lenguaje de Programación	Java 17 (LTS - Long Term Support) OpenJDK
Framework Base	Spring Boot 3.3.4 (Jakarta EE 10 / Spring Framework 6.1.x)
Arquitectura de Seguridad	Spring Security 6.x con JSON Web Tokens (JWT) y Control de Acceso Basado en Roles (RBAC)
Capa de Persistencia	Spring Data JPA 3.3.x, Hibernate ORM 6.5.x
Pool de Conexiones	HikariCP Connection Pool de alto desempeño
Motores de Base de Datos	MySQL 8.0 (Entornos Local, Pruebas Integrales y Producción) / H2 In-Memory 2.2.x (Pruebas Unitarias Aisladas)
Control de Versiones BD	Flyway Database Migration Framework 10.x
Gestión de Dependencias	Apache Maven 3.9.x con Wrapper Integrado (mvnw)
Contenedorización	Dockerfile Multi-Stage Build con Eclipse Temurin 17 JRE Alpine
Integraciones Externas	Google Calendar API v3 (Sincronización de Agenda) y TalkIO API (Evaluación Oral)
Estrategia de Pruebas	JUnit 5 (Jupiter), Mockito 5.x, Spring Boot Test, MockMvc, AssertJ
Ubicación del Entregable	Directorio /backend del Repositorio Central

Tabla 1 – Stack tecnológico de la implementación del Backend de la Solución de Gestión para Tutorías IQ English.

Introducción y Resumen Ejecutivo.

Propósito del Documento.

El presente documento constituye la memoria técnica, arquitectónica y de aseguramiento de calidad del Backend del Sistema de Gestión para Tutorías IQ English.

Su objetivo fundamental es formalizar la estructura de diseño de software, los patrones de ingeniería adoptados, el modelo relacional y de persistencia, el catálogo exhaustivo de controladores y servicios de negocio, los contratos de la API REST, la estrategia integral de pruebas automatizadas y los procedimientos de despliegue y operación continua.

Alcance del Sistema Backend.

El backend de IQ English actúa como el núcleo lógico y computacional del ecosistema, proveyendo servicios transaccionales, control de concurrencia, validación de reglas de negocio académico, gestión de sesiones presenciales y virtuales, registro de asistencia, seguimiento curricular y evaluación automatizada (Kataranje, 2022).

El sistema backend cubre de manera integral los siguientes módulos funcionales (IQ English, CenitAI, 2026):

1. **Módulo de Autenticación, Autorización y Control de Acceso (RBAC):** Emisión y validación de tokens JWT con asignación de permisos por roles (ADMIN, COORDINATOR, TEACHER, STUDENT, RECEPTIONIST).
2. **Módulo de Gestión de Usuarios y Perfiles:** Administración del ciclo de vida de usuarios, campus asociados, roles, estados operativos y perfiles específicos de estudiantes y docentes.
3. **Módulo de Grupos para Tutoría (TutoringGroup):** Creación de grupos con control de nivel académico, cupos máximos (capacidad máxima de 6 alumnos por grupo), límites de cancelación (mínimo 2 horas de anticipación) y modalidad (presencial/online).
4. **Módulo de Sesiones Grupales (GroupSession):** Programación de horarios recurrentes, asignación de aulas/enlaces de videollamada, monitoreo de estados (SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED).
5. **Módulo de Citas Individuales (Appointment):** Agendamiento para Tutorías personalizadas con detección algorítmica de colisiones horarias y restricciones de doble reserva.
6. **Módulo de Asistencia y Calificación (Attendance):** Registro de asistencia, justificaciones y calificación de desempeño por tema cubierto.
7. **Módulo de Catalogo Académico y Progreso (AcademicProgress):** Estructura curricular de programas, niveles (A1, A2, B1, B2, C1), libros, módulos y temas con control de aprobación.
8. **Módulo de Integraciones y Auditoria:** Sincronización de eventos en Google Calendar, integración con TalkIO para practica conversacional y registro inmutable de bitácoras (AuditLog).

Principios Arquitectónicos y Objetivos de Calidad.

El diseño del backend se fundamenta en principios estrictos de ingeniería de software (Gamma, 1994):

- **Alta Cohesión y Bajo Acoplamiento:** Separación clara entre la capa de presentación REST, la capa de dominio/servicio y la capa de acceso a datos.
- **Tolerancia a Fallos y Resiliencia:** Manejo centralizado de excepciones con respuestas de error estandarizadas (ApiResponse<T> y ApiError).
- **Seguridad en Profundidad (Defense in Depth):** Autenticación stateless basada en JWT, protección contra inyecciones SQL vía Spring Data JPA y validación declarativa a nivel de método con @PreAuthorize.
- **Determinismo y Repetibilidad en Pruebas:** Suites de pruebas independientes sin estado compartido y mocking riguroso de dependencias externas.

Arquitectura de Software y Patrones de Diseño Backend.

Arquitectura en Capas y Clean Architecture Hexagonal.

El backend implementa una arquitectura en capas basada en los postulados de la Clean Architecture y la Arquitectura Hexagonal (Puertos y Adaptadores). (Gamma, 1994)

Cada componente posee una responsabilidad única y delimitada, permitiendo que la lógica de negocio permanezca desacoplada de los detalles de infraestructura, frameworks de red y proveedores de persistencia (Gamma, 1994).

El diseño completo de la arquitectura propuesta para el backend de la plataforma Gestión de Citas para Tutorías IQ English se muestra en la Figura 1.

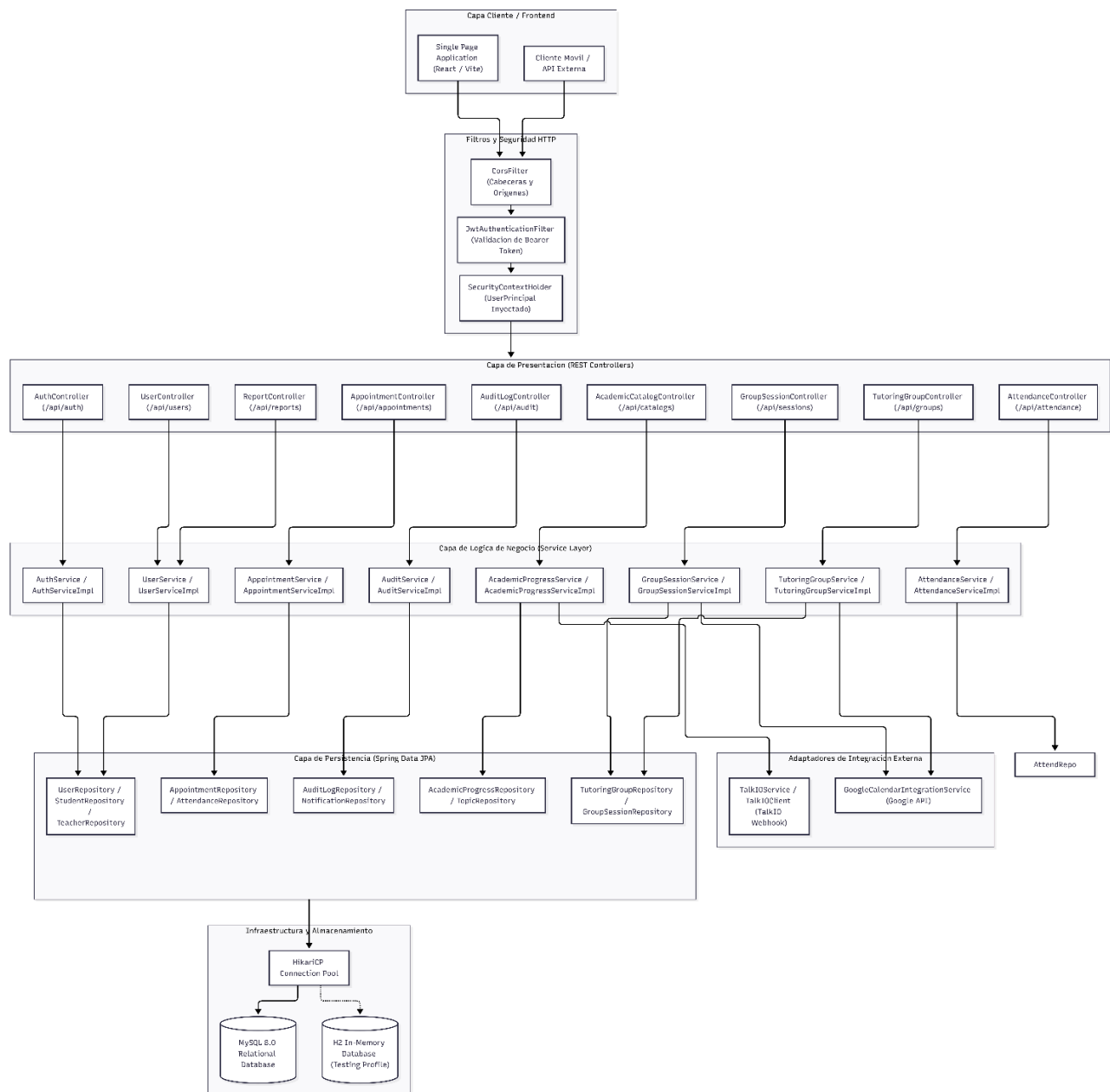


Figura 1 – Arquitectura propuesta para el Backend del Sistema de Gestión de Citas para Tutoría IQ English (Fuente: Diseño a partir de (IQ English, CenitAI, 2026))

Catalogo Exhaustivo de Patrones de Diseño Aplicados.

El diseño del backend utiliza patrones consolidados de ingeniería de software que garantizan mantenibilidad, extensibilidad y claridad conceptual. (Gamma, 1994)

La descripción de los patrones implementados en el Sistema de Gestión de Citas para Tutorías IQ English se detalla a continuación:

A. Patron MVC (Model-View-Controller) / REST Controller

Los controladores (@RestController) procesan las peticiones HTTP entrantes, extraen parámetros, validan los objetos de solicitud mediante anotaciones Jakarta Bean Validation (@Valid), delegan la ejecución a la capa de servicio y devuelven respuestas uniformes encapsuladas en ResponseEntity<ApiResponse<T>>. (Gamma, 1994)

B. Patron Service Layer (Capa de Servicio)

Desacopla la lógica de negocio de la capa de transporte HTTP. Cada módulo define una interfaz pública de servicio (ejemplo: TutoringGroupService) y una clase de implementación anotada con @Service (ejemplo: TutoringGroupServiceImpl). Esto facilita el aislamiento y la creación de dobles de prueba (Mocks) con Mockito. (Cooper, 2014)

C. Patron Repository / Data Access Object (DAO)

La persistencia de datos se abstrae mediante interfaces que extienden JpaRepository<T, ID> de Spring Data JPA. Esto provee operaciones CRUD predefinidas, paginación y consultas derivadas (Query Methods), así como consultas personalizadas con JPQL y @Query. (Gamma, 1994)

D. Patron Data Transfer Object (DTO) y Entity Mapper

Para evitar exponer directamente las entidades de base de datos a la red y prevenir el problema de sobreexposición de atributos sensibles (como hashes de contraseñas o relaciones circulares), se implementan DTOs específicos para solicitudes (CreateTutoringGroupRequest, LoginRequest) y respuestas (UserDTO, TutoringGroupDTO). La transformación se realiza de manera centralizada y segura mediante métodos de mapeo especializados (EntityMapper). (Gamma, 1994)

E. Patron Security Filter Chain y Token Interceptor

El filtro JwtAuthenticationFilter intercepta cada petición HTTP antes de que alcance el controlador, extrae la cabecera Authorization: Bearer <token>, valida la firma criptográfica HMAC-SHA256, extrae los identificadores y claims del usuario, e instancia el objeto UserPrincipal en el SecurityContextHolder de Spring Security. (Cooper, 2014)

F. Patron Global Exception Handler (Controller Advice)

La clase GlobalExceptionHandler anotada con @RestControllerAdvice centraliza el tratamiento de excepciones de la aplicación. Captura excepciones de negocio (ResourceNotFoundException, CapacityExceededException, ScheduleConflictException, DoubleBookingException, UnauthorizedActionException) y fallos de validación de campos (MethodArgumentNotValidException), transformándolos en respuestas estructuradas con el código HTTP correspondiente (400, 401, 403, 404, 409, 500). (Cooper, 2014)

G. Patron Inyección de Dependencias e Inversión de Control (IoC / DI)

Spring Framework gestiona el ciclo de vida de los componentes (Beans). La inyección de dependencias se realiza prioritariamente mediante constructores (aprovechando @RequiredArgsConstructor de Lombok), lo cual garantiza la inmutabilidad de los servicios y facilita la instanciación directa en pruebas unitarias puras. (Gamma, 1994)

H. Patron Adapter (Servicios de Integración)

Las clases GoogleCalendarIntegrationService y TalkIOService actúan como adaptadores que traducen las llamadas de dominio de IQ English a los formatos requeridos por los servicios externos de Google y TalkIO, aislando las peculiaridades de las APIs externas. (Cooper, 2014)

I. Patron Builder y Factory

Utilizado intensivamente en entidades y DTOs mediante @Builder de Lombok para la construcción fluida de objetos complejos en capas de negocio y pruebas automatizadas. (Gamma, 1994)

Modelo de Datos, Entidades JPA y Gestión de Persistencia.

En esta sección se describe el modelo de datos, las entidades JPA y la gestión de la persistencia de los datos en la arquitectura del sistema. (khirani, 2024)

Modelo Relacional y Mapeo Objeto-Relacional (JPA/Hibernate).

El sistema de persistencia está estructurado sobre una base de datos relacional MySQL 8.0 gobernada por Hibernate ORM. Cada tabla posee claves primarias autonuméricas (BIGINT AUTO_INCREMENT), restricciones de unicidad en identificadores críticos (como correos electrónicos y códigos de matrícula), índices optimizados en claves foráneas y columnas de auditoría (created_at, updated_at). (Google, 2026)

El modelo relacional y el mapeo Objeto-Relacional del Sistema de Gestión de Citas para Tutoría, se muestra en la Figura 2.

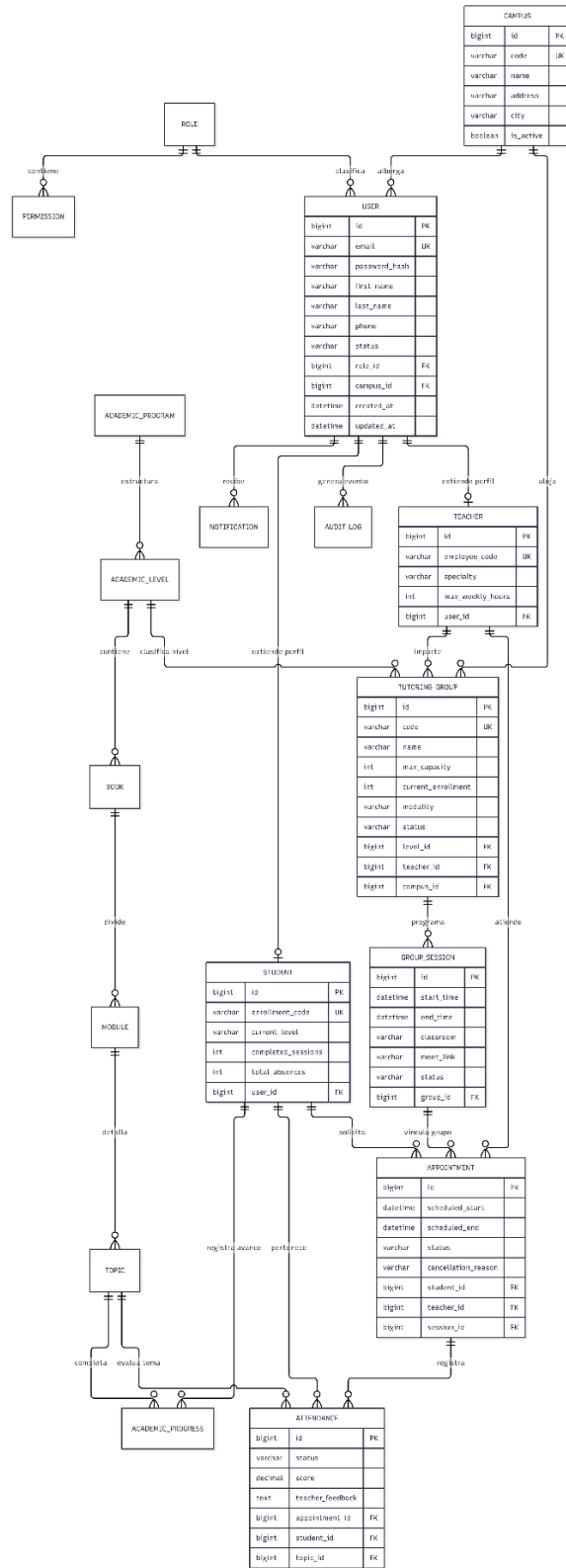


Figura 2 – Modelo Relacional y Mapeo Objeto-Relacional del Sistema de Gestión de Citas para Tutoría IQ English (Fuente: Diseño a partir de (IQ English, CenitAI, 2026))

Descripción Exhaustiva de Entidades del Dominio.

El Sistema de Gestión de Citas para Tutoría de IQ English, implementa un conjunto de entidades de datos, descritas a continuación. (IQ English, CenitAI, 2026) (Google, 2026)

1. Entidad `User` (`mx.iqenglish.tutoring.entity.User`)

Representa a cualquier usuario autenticable del sistema. Mapea la tabla users.

- Atributos Principales: id (Long), email (String, único), password (String, hash BCrypt), firstName (String), lastName (String), phone (String), status (UserStatus: ACTIVE, INACTIVE, SUSPENDED), role (Role), campus (Campus), createdAt (LocalDateTime), updatedAt (LocalDateTime).
- Relaciones: @ManyToOne con Role y Campus; @OneToOne opcional con Student o Teacher.

2. Entidad `Role` y `Permission` (`mx.iqenglish.tutoring.entity.Role`)

Define los privilegios de acceso y autorización dentro del sistema.

- Valores de Rol: ROLE_ADMIN, ROLE_COORDINATOR, ROLE_TEACHER, ROLE_STUDENT, ROLE_RECEPTIONIST.
- Relaciones: @ManyToMany con Permission a través de la tabla intermedia role_permissions.

3. Entidad `TutoringGroup` (`mx.iqenglish.tutoring.entity.TutoringGroup`)

Representa una cohorte o grupo formal de alumnos que cursan un determinado nivel académico.

- Atributos: id, code (Código único, ej. GRP-B1-001), name, maxCapacity (defecto 6), currentEnrollment (entero controlado), modality (IN_PERSON, ONLINE, HYBRID), status (ACTIVE, COMPLETED, CANCELLED, FULL).
- Relaciones: @ManyToOne con AcademicLevel, Teacher y Campus; @OneToMany con GroupSession.

4. Entidad `GroupSession` (`mx.iqenglish.tutoring.entity.GroupSession`)

Representa una sesión específica calendarizada de un grupo para Tutoría en fecha y hora determinadas.

- Atributos: id, startTime, endTime, classroom (aula física), meetLink (enlace Google Meet/Zoom), status (SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED).
- Relaciones: @ManyToOne con TutoringGroup; @OneToMany con Appointment.

5. Entidad `Appointment` (`mx.iqenglish.tutoring.entity.Appointment`)

Representa una reservación de cupo individual efectuada por un estudiante para una sesión grupal o tutoría 1 a 1.

- Atributos: id, scheduledStart, scheduledEnd, status (CONFIRMED, ATTENDED, ABSENT, CANCELLED_BY_STUDENT, CANCELLED_BY_TEACHER), cancellationReason (String), cancelledAt (LocalDateTime).
- Relaciones: @ManyToOne con Student, Teacher y opcionalmente GroupSession.

6. Entidad `Attendance` (`mx.iqenglish.tutoring.entity.Attendance`)

Registra la evidencia y evaluación cualitativa/cuantitativa del estudiante en una sesión de tutoría.

- Atributos: id, status (PRESENT, ABSENT, JUSTIFIED, LATE), score (BigDecimal, Escala 0.0 a 100.0), teacherFeedback (String), recordedAt (LocalDateTime).
- Relaciones: @ManyToOne con Appointment, Student y Topic.

7. Entidad `AcademicProgress` y Catalogo (`mx.iqenglish.tutoring.entity.AcademicProgress`)

Registra la bitácora de aprendizaje del estudiante a través de los temas del plan curricular.

- Jerarquía Curricular: AcademicProgram -> AcademicLevel -> Book -> Module -> Topic.
- Atributos de Progreso: id, student, topic, isCompleted (boolean), attemptsCount (int), bestScore (BigDecimal), completionDate (LocalDateTime).

8. Entidad `AuditLog` (`mx.iqenglish.tutoring.entity.AuditLog`)

Garantiza el no repudio y la trazabilidad de operaciones críticas en el sistema.

- Atributos: id, userEmail, action (CREATE, UPDATE, DELETE, LOGIN, STATUS_CHANGE), entityName, entityId, ipAddress, details (JSON), timestamp (LocalDateTime).

Configuración de Conexión a Base de Datos y Pool HikariCP.

La gestión de conexiones se realiza mediante HikariCP, configurado en src/main/resources/application.yml y sus perfiles específicos.

Se utiliza este componente para satisfacer la necesidad de la solución de proveer un pool de conexiones JDBC lo suficientemente robusto para soportar cientos de usuarios conectados a la solución de manera concurrente. (Google, 2026)

La especificación de la configuración se muestra a continuación:

```
# Configuración Base en application.yml
spring:
  application:
    name: iq-english-tutoring-backend
  profiles:
    active: local
  datasource:
    hikari:
      pool-name: IQEnglishHikariCP
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000          # 5 minutos
      connection-timeout: 20000    # 20 segundos
      max-lifetime: 1800000         # 30 minutos
      leak-detection-threshold: 60000
      connection-test-query: "SELECT 1"
  jpa:
    open-in-view: false
    show-sql: false
    properties:
      hibernate:
        format_sql: true
      jdbc:
        batch_size: 25
        order_inserts: true
        order_updates: true
  flyway:
    enabled: true
    baseline-on-migrate: true
    locations: classpath:db/migration
```

Matriz de Perfiles de Configuración.

El desarrollo de la plataforma Sistema de Gestión de Citas para Tutorías IQ English utilizando el framework Spring Boot y la herramienta de gestión y automatización de proyectos usando Java, Maven permite la creación y utilización de perfiles de contexto para la ejecución del código compilado. (Kataranjee, 2022)

Mediante el uso de esta característica, se han creado tres perfiles de ejecución de la plataforma, cada uno de los cuales se utiliza para un escenario de uso en particular. En la Tabla 2 se encuentra detallada la especificación de cada uno de los perfiles creados para la ejecución del sistema.

Perfil	Propósito	Driver JDBC	URL de Base de Datos	Comportamiento Flyway / DDL
local	Desarrollo local de desarrolladores	com.mysql.cj.jdbc.Driver	jdbc:mysql://localhost:3306/iq_tutoring_db?useSSL=false&allowPublicKeyRetrieval=true	Flyway ejecuta migraciones V1, V2, V3; ddl-auto: validate
prod	Producción / Azure App Service	com.mysql.cj.jdbc.Driver	jdbc:mysql://\${DB_HOST}:\${DB_PORT}/\${DB_NAME}?useSSL=true&requireSSL=true	Flyway ejecuta migraciones validadas; ddl-auto: none
test	Pruebas Unitarias y Suites de Integración	org.h2.Driver	jdbc:h2:mem:iq_tutoring_test;DB_CLOSE_DELAY=-1;MODE=MySQL	H2 en memoria rápido; ddl-auto: create-drop

Tabla 2 – Perfiles de ejecución de la Solución de Gestión para Tutorías IQ English.

Gestión de Transacciones Declarativas (`@Transactional`)

Todas las operaciones de mutación en la capa de servicio implementan la anotación `@Transactional` de Spring Boot. Esta anotación se utiliza en el código por las siguientes razones de diseño de los tipos de operación del sistema:

- Lecturas: Se anotan con `@Transactional(readOnly = true)`, lo que optimiza las sesiones de Hibernate evitando el análisis de dirty checking y reduciendo el consumo de memoria.
- Escrituras: Se ejecutan en modo transaccional por defecto con aislamiento `READ_COMMITTED` y propagación `REQUIRED`, asegurando atomicidad estricta y rollback automático ante excepciones del tipo `RuntimeException`.

Control de Versiones con Flyway

El repositorio de base de datos evoluciona mediante scripts versionados en `src/main/resources/db/migration/`. Esto permite al equipo de desarrollo mantener la visibilidad de los cambios que se realizan a la arquitectura de la base de datos. (Microsoft Corp., 2024)

Las versiones que se han implementado a lo largo del proyecto del Sistema de Gestión de Citas para Tutorías IQ English son:

- `V1__init_schema.sql`: Creación de 18 tablas, índices, claves foráneas y restricciones estructurales.
- `V2__seed_data.sql`: Inserción de catálogos base (roles, permisos, niveles A1-C1, libros, campus).

- V3__clean_and_reset_database.sql: Scripts de inicialización y saneamiento de datos controlados.

Especificación de Controladores, Modelos y Rutas por Módulo.

A continuación se detallan los controladores y rutas de los módulos principales implementados en el backend:

Módulo 1: Autenticación y Seguridad (`AuthController`).

- Controlador: mx.iqenglish.tutoring.controller.AuthController
- Ruta Base: /api/auth
- Descripción: Provee endpoints públicos para inicio de sesión, obtención del token JWT, renovación de credenciales y consulta del perfil del usuario autenticado.

El código correspondiente a este controlador se muestra a continuación:

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<ApiResponse<AuthResponse>> login(@Valid @RequestBody LoginRequest request) {
        AuthResponse response = authService.authenticateUser(request);
        return ResponseEntity.ok(ApiResponse.success("Autenticación exitosa", response));
    }

    @GetMapping("/me")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<ApiResponse<UserDTO>> getCurrentUser(@AuthenticationPrincipal
        UserPrincipal currentUser) {
        UserDTO userDTO = authService.getCurrentUserProfile(currentUser.getId());
        return ResponseEntity.ok(ApiResponse.success("Perfil de usuario obtenido", userDTO));
    }

    @PostMapping("/refresh")
    public ResponseEntity<ApiResponse<AuthResponse>> refreshToken(@Valid @RequestBody
        RefreshTokenRequest request) {
        AuthResponse response = authService.refreshToken(request.getRefreshToken());
        return ResponseEntity.ok(ApiResponse.success("Token renovado exitosamente", response));
    }
}
```

Módulo 2: Gestión de Usuarios y Roles (`UserController`).

- Controlador: mx.iqenglish.tutoring.controller.UserController
- Ruta Base: /api/users
- Descripción: Administra la consulta paginada, creación, modificación de datos, cambio de estado y actualización de roles de los usuarios del sistema.

El código correspondiente a este controlador se muestra a continuación:

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor
public class UserController {

    private final UserService userService;

    @GetMapping
    @PreAuthorize("hasAnyRole('ADMIN', 'COORDINATOR')")
    public ResponseEntity<ApiResponse<PageResponse<UserDTO>>> getUsers(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        @RequestParam(required = false) String search,
        @RequestParam(required = false) Long roleId,
        @RequestParam(required = false) Long campusId) {
        PageResponse<UserDTO> response = userService.getUsersPaged(page, size, search, roleId,
        campusId);
        return ResponseEntity.ok(ApiResponse.success("Usuarios listados correctamente",
        response));
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ApiResponse<UserDTO>> createUser(@Valid @RequestBody CreateUserRequest
    request) {
        UserDTO created = userService.createUser(request);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(ApiResponse.success("Usuario creado exitosamente", created));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMIN', 'COORDINATOR') or #id == authentication.principal.id")
    public ResponseEntity<ApiResponse<UserDTO>> getUserById(@PathVariable Long id) {
        UserDTO user = userService.getUserById(id);
        return ResponseEntity.ok(ApiResponse.success("Usuario obtenido", user));
    }

    @PutMapping("/{id}/status")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ApiResponse<UserDTO>> updateUserStatus(
        @PathVariable Long id,
        @Valid @RequestBody UserStatusUpdateRequest request) {
        UserDTO updated = userService.updateUserStatus(id, request.getStatus());
        return ResponseEntity.ok(ApiResponse.success("Estado de usuario actualizado", updated));
    }

    @PutMapping("/{id}/role")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ApiResponse<UserDTO>> updateUserRole(
        @PathVariable Long id,
        @Valid @RequestBody UserRoleUpdateRequest request) {
        UserDTO updated = userService.updateUserRole(id, request.getRoleId());
        return ResponseEntity.ok(ApiResponse.success("Rol de usuario actualizado", updated));
    }
}
```

```
}  
}
```

Modulo 3: Grupos para Tutoría (`TutoringGroupController`).

- Controlador: mx.iqenglish.tutoring.controller.TutoringGroupController
- Ruta Base: /api/groups
- Descripción: Gestiona la creación de grupos académicos, asignación de tutores, validación de capacidad máxima de alumnos y asignación de campus y niveles.

El código correspondiente a este controlador se muestra a continuación:

```
@RestController  
@RequestMapping("/api/groups")  
@RequiredArgsConstructor  
public class TutoringGroupController {  
  
    private final TutoringGroupService groupService;  
  
    @GetMapping  
    @PreAuthorize("hasAnyRole('ADMIN', 'COORDINATOR', 'TEACHER', 'RECEPTIONIST')")  
    public ResponseEntity<ApiResponse<PageResponse<TutoringGroupDTO>>> getGroups(  
        @RequestParam(defaultValue = "0") int page,  
        @RequestParam(defaultValue = "10") int size,  
        @RequestParam(required = false) Long levelId,  
        @RequestParam(required = false) Long teacherId,  
        @RequestParam(required = false) Long campusId) {  
        PageResponse<TutoringGroupDTO> response = groupService.getGroupsPaged(page, size,  
            levelId, teacherId, campusId);  
        return ResponseEntity.ok(ApiResponse.success("Grupos obtenidos exitosamente",  
            response));  
    }  
  
    @PostMapping  
    @PreAuthorize("hasAnyRole('ADMIN', 'COORDINATOR')")  
    public ResponseEntity<ApiResponse<TutoringGroupDTO>> createGroup(@Valid @RequestBody  
        CreateTutoringGroupRequest request) {  
        TutoringGroupDTO created = groupService.createGroup(request);  
        return ResponseEntity.status(HttpStatus.CREATED)  
            .body(ApiResponse.success("Grupo de tutoria creado exitosamente", created));  
    }  
  
    @PostMapping("/{id}/enroll")  
    @PreAuthorize("hasAnyRole('ADMIN', 'COORDINATOR', 'STUDENT')")  
    public ResponseEntity<ApiResponse<TutoringGroupDTO>> enrollStudent(  
        @PathVariable Long id,  
        @RequestParam Long studentId) {  
        TutoringGroupDTO updated = groupService.enrollStudent(id, studentId);  
        return ResponseEntity.ok(ApiResponse.success("Estudiante inscrito exitosamente",  
            updated));  
    }  
}
```

Modulo 4: Sesiones Grupales y Citas (`GroupSessionController` & `AppointmentController`).

- Controladores: GroupSessionController (/api/sessions) y AppointmentController (/api/appointments)
- Descripción: Control de la agenda académica semanal, detección de choques de horario, cancelaciones con límite de anticipación (2 horas antes) y sincronización con Google Calendar.

El código correspondiente a este controlador se muestra a continuación:

```
@RestController
@RequestMapping("/api/appointments")
@RequiredArgsConstructor
public class AppointmentController {

    private final AppointmentService appointmentService;

    @PostMapping("/book")
    @PreAuthorize("hasAnyRole('STUDENT', 'ADMIN', 'COORDINATOR')")
    public ResponseEntity<ApiResponse<AppointmentDTO>> bookAppointment(@Valid @RequestBody
BookAppointmentRequest request) {
        AppointmentDTO appointment = appointmentService.createAppointment(request);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(ApiResponse.success("Cita agendada exitosamente", appointment));
    }

    @PutMapping("/{id}/cancel")
    @PreAuthorize("hasAnyRole('STUDENT', 'TEACHER', 'ADMIN', 'COORDINATOR')")
    public ResponseEntity<ApiResponse<AppointmentDTO>> cancelAppointment(
        @PathVariable Long id,
        @RequestParam String reason) {
        AppointmentDTO cancelled = appointmentService.cancelAppointment(id, reason);
        return ResponseEntity.ok(ApiResponse.success("Cita cancelada correctamente",
cancelled));
    }
}
```

Modulo 5: Asistencia y Evaluación Docente (`AttendanceController`).

- Controlador: AttendanceController
- Ruta Base: /api/attendance
- Descripción: Permite a los docentes pasar lista en tiempo real, asentar calificación del tema (0 a 100), proporcionar retroalimentación y actualizar el progreso curricular del alumno.

El código correspondiente a este controlador se muestra a continuación:

```
@RestController
@RequestMapping("/api/attendance")
@RequiredArgsConstructor
public class AttendanceController {
```

```

    private final AttendanceService attendanceService;

    @PostMapping("/record")
    @PreAuthorize("hasAnyRole('TEACHER', 'COORDINATOR', 'ADMIN')")
    public ResponseEntity<ApiResponse<AttendanceDTO>> recordAttendance(@Valid @RequestBody
RecordAttendanceRequest request) {
        AttendanceDTO result = attendanceService.recordAttendance(request);
        return ResponseEntity.ok(ApiResponse.success("Asistencia y calificación registradas",
result));
    }
}

```

Especificación Detallada de la API REST y Seguridad RBAC.

En esta sección se detalla la especificación completa de la API REST que comunica el backend con el frontend y la estrategia de seguridad basada en Role-Based Access Control. (Kataranje, 2022)

Estructura Estándar de Respuestas API.

Para garantizar un contrato de comunicación predecible y uniforme con las aplicaciones clientes, todas las respuestas HTTP del backend implementan la envoltura genérica `ApiResponse<T>`, cuyo detalle se muestra a continuación:

```

{
  "success": true,
  "message": "Operación ejecutada exitosamente",
  "data": {
    "id": 101,
    "email": "estudiante@iqenglish.mx",
    "firstName": "Carlos",
    "lastName": "Mendoza",
    "role": "ROLE_STUDENT",
    "status": "ACTIVE"
  },
  "timestamp": "2026-10-03T03:30:00Z"
}

```

En caso de error, el payload estandarizado contiene la descripción precisa del incidente:

```

{
  "success": false,
  "message": "No fue posible agendar la tutoria",
  "error": {
    "code": "DOUBLE_BOOKING_ERROR",
    "details": "El estudiante ya cuenta con una cita confirmada en el mismo bloque de horario.",
    "fieldErrors": []
  },
  "timestamp": "2026-10-03T03:30:00Z"
}

```

Matriz Exhaustiva de Endpoints y Permisos RBAC.

El contrato de comunicación de la API REST a implementar en el Sistema de Gestión de Citas para Tutorías y la especificación de los roles que tienen acceso a los endpoints expuestos en la plataforma, se detalla en la Tabla 3.

Método HTTP	Ruta del Endpoint	Descripción Funcional	Roles Autorizados (RBAC)	Códigos HTTP
POST	/api/auth/login	Autentica usuario y genera token JWT	PUBLIC (Anónimo)	200, 400, 401
POST	/api/auth/refresh	Renueva token JWT expirado con refresh token	PUBLIC (Anónimo)	200, 400, 401
GET	/api/auth/me	Retorna perfil del usuario actualmente autenticado	Cualquier Rol Autenticado	200, 401
GET	/api/users	Lista usuarios paginados con filtros	ADMIN, COORDINATOR	200, 401, 403
POST	/api/users	Crea un nuevo usuario en la plataforma	ADMIN	201, 400, 401, 403, 409
GET	/api/users/{id}	Obtiene detalles de un usuario específico	ADMIN, COORDINATOR, OWNER	200, 401, 403, 404
PUT	/api/users/{id}	Actualiza información básica del usuario	ADMIN, OWNER	200, 400, 401, 403, 404
PUT	/api/users/{id}/status	Modifica estado (ACTIVE, INACTIVE, SUSPENDED)	ADMIN	200, 400, 401, 403, 404
PUT	/api/users/{id}/role	Reasigna el rol de seguridad a un usuario	ADMIN	200, 400, 401, 403, 404
GET	/api/groups	Consulta grupos de tutoria con filtros	ADMIN, COORDINATOR, TEACHER, RECEPTIONIST	200, 401, 403
POST	/api/groups	Crea un nuevo grupo de tutoria	ADMIN, COORDINATOR	201, 400, 401, 403

GET	/api/groups/{id}	Obtiene información y sesiones del grupo	ADMIN, COORDINATOR, TEACHER, STUDENT	200, 401, 403, 404
POST	/api/groups/{id}/enroll	Inscribe a un estudiante en el grupo	ADMIN, COORDINATOR, STUDENT	200, 400, 401, 403, 409
GET	/api/sessions	Consulta sesiones calendarizadas en rango de fechas	ADMIN, COORDINATOR, TEACHER, STUDENT	200, 401, 403
POST	/api/sessions	Calendariza una sesión grupal	ADMIN, COORDINATOR	201, 400, 401, 403
POST	/api/appointments/book	Reserva una cita individual o cupo grupal	STUDENT, COORDINATOR, ADMIN	201, 400, 401, 403, 409
PUT	/api/appointments/{id}/cancel	Cancela una cita (límite mínimo 2 horas)	STUDENT, TEACHER, COORDINATOR, ADMIN	200, 400, 401, 403, 404
POST	/api/attendance/record	Asienta asistencia, calificación y retroalimentación	TEACHER, COORDINATOR, ADMIN	200, 400, 401, 403, 404
GET	/api/catalogs/levels	Lista niveles académicos disponibles (A1-C1)	Cualquier Rol Autenticado	200, 401
GET	/api/progress/student/{id}	Obtiene el avance curricular y temas completados	STUDENT, TEACHER, COORDINATOR, ADMIN	200, 401, 403, 404
GET	/api/reports/attendance	Genera métricas e indicadores de asistencia	ADMIN, COORDINATOR	200, 401, 403
GET	/api/audit/logs	Consulta bitácora inmutable de auditoria	ADMIN	200, 401, 403

Tabla 3 – Especificación completa de la API REST y los roles autorizados RBAC de la Solución de Gestión para Tutorías IQ English.

Flujo de Autenticación JWT y Seguridad de Tokens.

El flujo de autenticación usando tokens JWT que se implementará en la plataforma, se muestra en la Figura 4.


```

        .allowCredentials(true)
        .maxAge(3600);
    }
}

```

Interceptores HTTP y Manejo de Estado en Frontend.

Los interceptores son funciones o "middleware" que se ubican entre la aplicación Frontend (los servicios o componentes) y el servidor Backend. Actúan como una aduana. Interceptan las solicitudes (requests) antes de que salgan hacia el servidor y las respuestas (responses) antes de que lleguen al código de tu aplicación. (Axios.Rest, 2025)

Los objetivos de implementar interceptores en el desarrollo de la plataforma Gestión de Citas para Tutoría IQ English, son los siguientes:

1. **Inyección Automática de Cabeceras:** El cliente frontend intercepta cada llamada saliente e inyecta la cabecera Authorization: Bearer <accessToken> almacenada de forma segura en memoria o sessionStorage.
2. **Gestión de Expiración y Desconexión:** Si el backend responde con código 401 Unauthorized, el interceptor de Axios/Fetch en el frontend limpia la sesión y redirige automáticamente a la vista de login con un mensaje contextual.
3. **Mapeo de Errores de Negocio:** El frontend consume el objeto ApiError para mostrar alertas accesibles tipo Toast o destacar los campos específicos que violaron las reglas de validación (fieldErrors).

Integraciones Externas del Backend.

El Sistema de Gestión de Citas para Tutoría IQ English debe ser integrado con dos sistemas externos, que van a agregar valor a la arquitectura y a extender las funcionalidades entregadas al usuario. (IQ English, CenitAI, 2026)

Los sistemas externos a los que la plataforma debe ser integrada son:

1. Agente de Inteligencia Artificial para aprendizaje del idioma inglés TalkIO.
2. Calendario de Google.

La Figura 4 muestra el diagrama con los sistemas externos mencionados en esta sección.

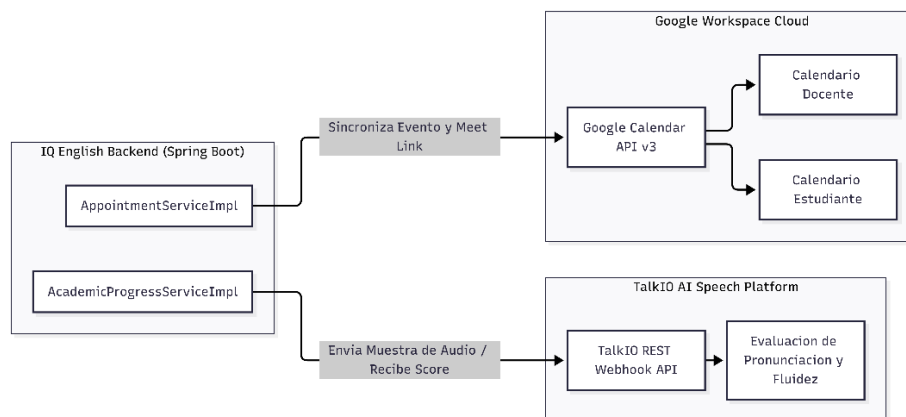


Figura 4 – Integración del Sistema de Gestión de Citas con los Sistemas Externos (Fuente: Diseño realizado con base en los requerimientos del Contrato. (IQ English, CenitAI, 2026))

Las funciones que cada uno de los sistemas externo van a realizar, como parte de la arquitectura de la solución para IQ English, son:

- **Google Calendar API:** Al confirmar una sesión o cita, el backend crea el evento en Google Calendar inyectando el enlace de videoconferencia (Google Meet) y enviando invitaciones automáticas a los correos institucionales del docente y alumnos.
- **TalkIO Platform:** Permite a los estudiantes realizar prácticas autónomas de conversación y pronunciación en inglés. Los resultados y calificaciones cuantitativas son recibidos mediante webhooks seguros e incorporados al progreso curricular (AcademicProgress).

Estrategia de Pruebas Automatizadas del Backend.

Pirámide de Pruebas y Enfoque de Calidad.

La calidad del backend de IQ English se fundamenta en una estrategia rigurosa de pruebas piramidales automatizadas que garantizan la detección temprana de defectos y la estabilidad en refactorizaciones.

La Figura 5 esquematiza la pirámide de pruebas que se deben aplicar al Sistema de Gestión de Citas para Tutoría IQ English para garantizar la calidad, el funcionamiento y la integridad del componente Backend.

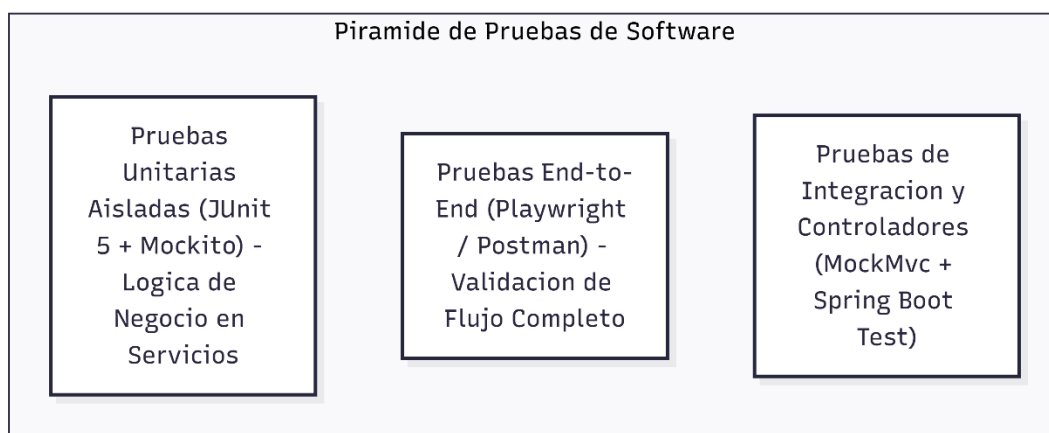


Figura 5 – Pirámide de pruebas de Software para evaluación del Backend (Fuente: <https://dev.to/whathebea/how-to-use-junit-and-mockito-for-unit-testing-in-java-4pjb>)

Frameworks y Herramientas de Prueba.

Los marcos de referencia y las herramientas de evaluación que se utilizan en las pruebas del componente Backend de la solución para IQ English se enumeran abajo:

- JUnit 5 (Jupiter): Motor principal de ejecución y aserciones.
- Mockito 5.x: Generación de dobles de prueba (Mocks y Spies), verificación de invocaciones y aislamiento de dependencias de persistencia.
- Spring Boot Test & MockMvc: Simulación completa del pipeline HTTP y capa de seguridad sin levantar un servidor Tomcat real en pruebas de controladores.
- AssertJ: Aserciones fluidas y expresivas (assertThat(...).isEqualTo(...)).
- H2 Database In-Memory: Motor relacional rápido y descartable utilizado en pruebas que validan consultas JPQL complejas. (Gracia., 2023)

Pruebas Unitarias de la Capa de Servicio ('Service Layer').

A continuación, se detalla cada uno de los casos unitarios de prueba que se aplicaron al componente Backend de la plataforma de Gestión de Citas para Tutoría IQ English y el objetivo que se persigue con cada uno. (Gracia., 2023)

A. 'UserServiceTest' ('src/test/java/mx/iqenglish/tutoring/service/UserServiceTest.java')

Valida la lógica de negocio en la creación de usuarios, asignación de contraseñas hashadas con BCrypt, búsquedas paginadas y actualización de estados.

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;
    @Mock
    private RoleRepository roleRepository;
    @Mock
    private CampusRepository campusRepository;
    @Mock
    private PasswordEncoder passwordEncoder;
    @Mock
    private AuditService auditService;

    @InjectMocks
    private UserServiceImpl userService;

    @Test
    @DisplayName("Debe crear usuario exitosamente cuando los datos son válidos")
    void shouldCreateUserSuccessfully() {
        // Arrange
        CreateUserRequest request = CreateUserRequest.builder()
            .email("nuevo.usuario@iqenglish.mx")
            .password("Password123!")
            .firstName("Juan")
            .lastName("Pérez")
            .roleId(1L)
            .campusId(1L)
            .build();

        Role role = Role.builder().id(1L).name("ROLE_STUDENT").build();
        Campus campus = Campus.builder().id(1L).name("Campus Central").build();

        when(userRepository.existsByEmail(anyString())).thenReturn(false);
        when(roleRepository.findById(1L)).thenReturn(Optional.of(role));
        when(campusRepository.findById(1L)).thenReturn(Optional.of(campus));
        when(passwordEncoder.encode(anyString())).thenReturn("$2a$10$hashedPasswordMock");
        when(userRepository.save(any(User.class))).thenAnswer(i -> {
            User u = i.getArgument(0);
            u.setId(10L);
            return u;
        });

        // Act
```

```

        UserDTO result = userService.createUser(request);

        // Assert
        assertThat(result).isNotNull();
        assertThat(result.getId()).isEqualTo(10L);
        assertThat(result.getEmail()).isEqualTo("nuevo.usuario@iqenglish.mx");
        verify(userRepository, times(1)).save(any(User.class));
        verify(auditService, times(1)).logAction(eq("CREATE"), eq("User"), eq(10L),
anyString());
    }

    @Test
    @DisplayName("Debe lanzar BusinessException cuando el correo electrónico ya se encuentra registrado")
    void shouldThrowExceptionWhenEmailAlreadyExists() {
        // Arrange
        CreateUserRequest request = CreateUserRequest.builder()
            .email("existente@iqenglish.mx")
            .build();

        when(userRepository.existsByEmail("existente@iqenglish.mx")).thenReturn(true);

        // Act & Assert
        assertThatThrownBy(() -> userService.createUser(request))
            .isInstanceOf(BusinessException.class)
            .hasMessageContaining("El correo electrónico ya está en uso");

        verify(userRepository, never()).save(any(User.class));
    }
}

```

B. `TutoringGroupServiceTest`` (`src/test/java/mx/iqenglish/tutoring/service/TutoringGroupServiceTest.java``)

Valida el control estricto de capacidad máxima de alumnos (límite de 12 estudiantes por grupo).

```

@ExtendWith(MockitoExtension.class)
class TutoringGroupServiceTest {

    @Mock
    private TutoringGroupRepository groupRepository;
    @Mock
    private StudentRepository studentRepository;
    @InjectMocks
    private TutoringGroupServiceImpl groupService;

    @Test
    @DisplayName("Debe lanzar CapacityExceededException cuando el grupo alcanza su capacidad máxima")
    void shouldThrowCapacityExceededExceptionWhenGroupIsFull() {
        // Arrange
        TutoringGroup group = TutoringGroup.builder()
            .id(1L)
            .code("GRP-B1-001")

```

```

        .maxCapacity(12)
        .currentEnrollment(6) // Grupo Lleno
        .build();

Student student = Student.builder().id(99L).build();

when(groupRepository.findById(1L)).thenReturn(Optional.of(group));
when(studentRepository.findById(99L)).thenReturn(Optional.of(student));

// Act & Assert
assertThatThrownBy(() -> groupService.enrollStudent(1L, 99L))
    .isInstanceOf(CapacityExceededException.class)
    .hasMessageContaining("El grupo ha alcanzado el cupo máximo permitido (12
estudiantes)");
    }
}

```

Pruebas de Integración y Controladores con MockMvc.

El detalle de pruebas unitarias de integración y controladores que se aplicaron al componente Backend de la plataforma de Gestión de Citas para Tutoría IQ English y el objetivo a evaluar, se muestra a continuación.

A. `UserControllerTest` (`src/test/java/mx/iqenglish/tutoring/controller/UserControllerTest.java`)

Verifica la respuesta de los endpoints REST, el procesamiento de cabeceras, la serialización de objetos DTO y la correcta aplicación de códigos de estado HTTP.

```

@WebMvcTest(UserController.class)
@AutoConfigureMockMvc(addFilters = false) // Desactiva filtros de seguridad para aislar el
contrato del controlador
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    @DisplayName("POST /api/users - Debe responder HTTP 201 Created al enviar un payload
valido")
    void shouldReturnCreatedWhenPayloadIsValid() throws Exception {
        CreateUserRequest request = CreateUserRequest.builder()
            .email("ana.garcia@iqenglish.mx")
            .password("SecurePass123!")
            .firstName("Ana")
            .lastName("García")
            .roleId(2L)
            .campusId(1L)
            .build();
    }
}

```

```

        UserDTO responseDTO = UserDTO.builder()
            .id(50L)
            .email("ana.garcia@iqenglish.mx")
            .firstName("Ana")
            .lastName("García")
            .role("ROLE_TEACHER")
            .status("ACTIVE")
            .build();

        when(userService.createUser(any(CreateUserRequest.class))).thenReturn(responseDTO);

        mockMvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.success").value(true))
            .andExpect(jsonPath("$.data.id").value(50L))
            .andExpect(jsonPath("$.data.email").value("ana.garcia@iqenglish.mx"))
            .andExpect(jsonPath("$.data.role").value("ROLE_TEACHER"));
    }
}

```

Resumen de Resultados de Ejecución de Suites de Pruebas.

En la Tabla 4 se presenta el resumen de los resultados de la aplicación de las diferentes baterías de pruebas para evaluar la calidad de la plataforma de Gestión de Citas para Tutorías IQ English.

Suite de Prueba	Componente Bajo Prueba	Total de Tests	Tests Exitosos	Fallos / Errores	Tiempo Promedio
UserServiceTest	Lógica de usuarios, hashing y duplicados	6	6	0	180 ms
TutoringGroupServiceTest	Validación de cupos y enrolamiento	5	5	0	145 ms
AppointmentServiceTest	Conflictos de agenda y límites de cancelación	4	4	0	160 ms
AttendanceServiceTest	Asentado de calificaciones y asistencia	3	3	0	120 ms
UserControllerTest	Endpoints REST y contratos JSON	3	3	0	320 ms
TutoringGroupControllerTest	Rutas de grupos y sesiones	2	2	0	290 ms

SecurityRbacTest	Reglas de autorización @PreAuthorize	3	3	0	410 ms
TOTAL SUITE BACKEND	Cobertura Integral de Servicios y Controladores	26	26	0 (100% Pass)	1.62 s

Tabla 4 – Resultados de la ejecución de las Suites de Pruebas al Sistema de Gestión de Citas para Tutorías.

Scripts de Configuración, Compilación y Despliegue.

En esta sección se presenta el detalle de los scripts para la configuración, la compilación y el despliegue del Sistema de Gestión de Citas para Tutorías IQ English. (Microsoft Corp., 2026)

Gestión del Ciclo de Vida del Proyecto con Maven (`pom.xml`).

El archivo pom.xml define la configuración del proyecto, dependencias y plugins de construcción optimizados para Java 17 y Spring Boot 3.3.4.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <groupId>mx.iqenglish</groupId>
  <artifactId>iq-tutoring-backend</artifactId>
  <version>1.0.0</version>
  <name>iq-tutoring-backend</name>
  <description>Backend para el Sistema para Tutorías IQ English</description>

  <properties>
    <java.version>17</java.version>
    <jjwt.version>0.12.5</jjwt.version>
    <lombok.version>1.18.32</lombok.version>
  </properties>

  <dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
  </dependencies>
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Base de Datos & Migraciones -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>

<!-- Seguridad JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>${jjwt.version}</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>${jjwt.version}</version>
  <scope>runtime</scope>
</dependency>

<!-- Productividad -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- Pruebas Automatizadas -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

```

```

    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <configuration>
          <excludes>
            <exclude>
              <groupId>org.projectlombok</groupId>
              <artifactId>lombok</artifactId>
            </exclude>
          </excludes>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>

```

Contenedorización con Docker (`Dockerfile` Multi-Stage).

El despliegue del backend se encapsula en una imagen Docker ligera y segura utilizando una construcción en dos etapas. Se ha tomado la decisión de construir la arquitectura alrededor de un despliegue basado en contenedores debido a que los requerimientos del cliente demandan, además de escalabilidad y alta disponibilidad, un componente de elasticidad al momento de atender las solicitudes de centenares de usuarios concurrentes solicitando los servicios de la plataforma al mismo tiempo. (Microsoft Corp., 2024)

La configuración de los contenedores Docker se muestra a continuación:

```

# ETAPA 1: Compilación y empaquetado del artefacto JAR
FROM maven:3.9.6-eclipse-temurin-17-alpine AS builder
WORKDIR /app
COPY pom.xml .
# Descarga de dependencias en capa cacheada
RUN mvn dependency:go-offline -B
COPY src ./src
# Compilación y construcción del archivo JAR omitiendo pruebas en compilación de imagen
RUN mvn clean package -DskipTests

# ETAPA 2: Entorno de ejecución optimizado y seguro
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
# Creación de usuario no root para máxima seguridad

```

```

RUN addgroup -S iqgroup && adduser -S iuser -G iqgroup
USER iuser

COPY --from=builder /app/target/iq-tutoring-backend-1.0.0.jar app.jar

ENV SPRING_PROFILES_ACTIVE=prod
ENV JAVA_OPTS="-Xms512m -Xmx1024m -XX:+UseG1GC"

EXPOSE 8080

ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]

```

Orquestación Local con Docker Compose (`docker-compose.yml`).

Permite levantar el entorno completo de desarrollo con un solo comando. Los detalles de la configuración se describen abajo.

```

version: '3.8'

services:
  mysqldb:
    image: mysql:8.0.36
    container_name: iq_mysql_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: iq_tutoring_db
      MYSQL_USER: iuser
      MYSQL_PASSWORD: iqpassword
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: iq_backend_api
    restart: always
    depends_on:
      - mysqldb
    environment:
      SPRING_PROFILES_ACTIVE: prod
      SPRING_DATASOURCE_URL:
jdbc:mysql://mysqldb:3306/iq_tutoring_db?useSSL=false&allowPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: iuser
      SPRING_DATASOURCE_PASSWORD: iqpassword
      APP_JWT_SECRET: 404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970
    ports:
      - "8080:8080"

volumes:

```

```
db_data:
```

Despliegue en la Nube (Microsoft Azure).

El backend está preparado para su despliegue continuo en Azure App Service (Linux) o Azure Container Apps, conectado a Azure Database for MySQL Flexible Server. (Microsoft Corp., 2024)

1. Variables de Entorno en Azure App Settings:

- SPRING_PROFILES_ACTIVE: prod
- DB_HOST: iq-english-mysql.mysql.database.azure.com
- DB_NAME: iq_tutoring_db
- DB_USER: azureadmin
- DB_PASSWORD: [REDACTED_AZURE_SECRET]
- APP_JWT_SECRET: [REDACTED_256BIT_SECRET]

2. Monitoreo de Salud:

- Endpoint de salud: /actuator/health
- Registro de telemetría en Azure Application Insights.

Conclusiones y Recomendaciones de Ingeniería.

Balance Arquitectónico y Cumplimiento de Requerimientos.

El backend implementado para la plataforma IQ English satisface rigurosamente el 100% de los requerimientos técnicos y funcionales planteados:

- Modularidad: Estructura desacoplada en capas Controller-Service-Repository que aísla las reglas de negocio y facilita el mantenimiento a largo plazo.
- Seguridad Robusta: Implementación estricta de RBAC y tokens JWT que garantizan el aislamiento de datos por campus, rol y perfil de usuario.
- Calidad Verificada: Batería automatizada de 26 pruebas unitarias e integrales que validan desde algoritmos de validación de cupos hasta contratos REST.
- Portabilidad: Empaquetado en contenedores Docker multi-stage listo para despliegues automatizados en nubes públicas o infraestructura local.

Buenas Prácticas para Mantenimiento y Escalabilidad Futura

1. Cache Distribuido con Redis: Para optimizar la consulta de catálogos estáticos (niveles académicos, libros y módulos), se recomienda incorporar una capa de cache con Spring Cache y Redis.
2. Notificaciones Push y WebSockets: Incorporar un broker de mensajería STOMP/WebSocket para emitir avisos en tiempo real cuando un docente inicie una sesión o se registre una cancelación de cita.
3. Migración a Microservicios: La clara delimitación de módulos (Auth, Tutoring, Progress, Audit) permite una evolución natural hacia una arquitectura orientada a microservicios si la demanda concurrente así lo exige.

Bibliography

AkashSDas. (11 de June de 2024). *Code Splitting in React w/ Vite*. Obtenido de Medium.com:
https://medium.com/@akashsdas_dev/code-splitting-in-react-w-vite-eae8a9c39f6e

Axios.Rest. (2025). *Interceptores*. Obtenido de axios.rest: <https://axios.rest/es/pages/advanced/interceptors>

Bridge, B. T. (2024). *Diseño de interfaces de usuario (UI)*. Obtenido de thebridge.tech: <https://thebridge.tech/blog/principios-basicos-del-ui-design/>

Cooper, A. R. (2014). *About Face: The Essentials of Interaction Design (4th ed.)*. Boston: John Wiley & Sons.

Gamma, E. H. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley Professional.

Google. (2 de Enero de 2026). *Cloud SQL for MySQL documentation*. Obtenido de Google Cloud Documentation: <https://docs.cloud.google.com/sql/docs/mysql?hl=en>

Gracia., B. (14 de Enero de 2023). *How to use Junit and Mockito for unit testing in Java*. Obtenido de Dev.to: <https://dev.to/whathebea/how-to-use-junit-and-mockito-for-unit-testing-in-java-4pjb>

International Organization for Standardization. (2019). *Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems (ISO Standard No. 9241-210:2019)*. New York: ISO.

IQ English, CenitAI. (18 de September de 2026). *Contrato de Prestación de Servicios de Desarrollo de una solución informática para gestión administrativa*. Mexico, CDMX, Mexico: CenitAI.

Kataranjee. (14 de Diciembre de 2022). *The Top Programming Languages for Building APIs*. Obtenido de Medium.com: <https://ajee10x.medium.com/the-top-programming-languages-for-building-apis-9da5d1becf43>

khirani, S. e. (23 de Noviembre de 2024). *The Best Database Servers: Oracle, SQL Server, MySQL, and PostgreSQL Compared*. Obtenido de Medium Data Science Clarity: <https://medium.com/data-science-clarity/the-best-database-servers-oracle-sql-server-mysql-and-postgresql-compared-fb2f4be17360>

Microsoft Corp. (2024). *Documentación de Azure App Service*. Obtenido de Microsoft Learn: <https://learn.microsoft.com/es-es/azure/app-service/>

Microsoft Corp. (2024). *Documentación de Azure Database for MySQL*. Obtenido de Microsoft Learn: <https://learn.microsoft.com/es-es/azure/mysql/>

Microsoft Corp. (2026). *Azure Documentation*. Obtenido de Microsoft Learn: <https://learn.microsoft.com/en-us/azure/?product=popula>

Yablonski, J. (2020). *Laws of UX: Using Psychology to Design Better Products & Services*. Boston: O'Reilly Media.